Software Architecture and Design Specification

Student Record System

Tanmay T A (PES1UG24CS492) Uttam B S (PES1UG24CS508)

Vibha Vasisht (PES1UG24CS526)

Generated 2026-11-14

Contents

Revision History **2**

Approvals **2**

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Audience . . . 3
1.4 Definitions and Acronyms . . . 3

2 Document Overview **4**
2.1 How to Use This Document . . . 4
2.2 Related Documents . . . 4

3 Architecture **4**
3.1 Goals & Constraints . . . 4
3.2 Stakeholders & Concerns . . . 5
3.3 Component Diagram . . . 5
3.4 Component Descriptions . . . 5
3.5 Chosen Architecture Pattern and Rationale . . . 6
3.6 Technology Stack & Data Stores . . . 6
3.7 Risks & Mitigations . . . 6
3.8 Traceability to Requirements . . . 7
3.9 Security Architecture (STRIDE Threat Model) . . . 7

4 Design **8**
4.1 Design Overview . . . 8
4.2 UML Sequence Diagrams . . . 8
4.2.1 Sequence Diagram 1: Digitized Attendance Marking & 48-Hour Freeze . . . 8
4.2.2 Sequence Diagram 2: Weighted Rubric Grade Lifecycle . . . 9
4.3 API Design . . . 10

4.3.1 Component Interface 1: Attendance Marking Endpoint 10
4.3.2 Component Interface 2: Grade Lifecycle & Dispute Endpoint 10
4.4 Error Handling, Logging & Monitoring 11
4.5 UX Design 11
4.6 Open Issues & Next Steps 11

5 Appendices **11**
5.1 Glossary 11
5.2 References 12
5.3 Tools 12

Revision History

Version	Date	Author	Change Summary	Approval
1.0	10-10-2026	Tanmay T A, Uttam B S, Vibha Vasisht	Initial Software Architecture and Design (SAD) specification draft for Student Record System.	Reviewed
1.1	28-10-2026	Tanmay T A	Aligned the grade lifecycle design with SRS 1.1: the sequence diagram marks a dispute as once per grade and the design text states that a re-published grade is final.	Approved

Approvals

Role	Name	Signature / Email	Date
Course Instructor	Dr. Adithya Balasubramanyam	adithyab@pes.edu	28-10-2026

1 Introduction

1.1 Purpose

This document provides a comprehensive Software Architecture and Design (SAD) specification for the Student Record System. It details the high-level system decomposition, architectural styles, constituent components, data persistence strategies, sequence models, API interfaces, security threat models (STRIDE), and error handling mechanisms.

1.2 Scope

This specification covers the system's software architecture for:

- **Account Provisioning & Profile Services:** Administrative user provisioning and self-service profile management.
- **Course & Self-Enrollment Module:** Offering registration, capacity management, weighted rubric scheme normalization, and student self-enrollment (max 6 courses).
- **Schedule & Roll-Call Attendance:** Class schedule creation, digitized roll-call entry, hard 48-hour automated attendance freeze, and administrative override.
- **Weighted Rubric Grading Workflow:** Rubric-based evaluation, grade state transitions (`Draft` → `Submitted` → `Under Review` → `Published`), and regrade dispute workflows.
- **Analytics & Summary Reporting:** Student GPA calculation, class grade distributions, and institution-level summary metrics.

Out of scope: Mail automation/SMTP delivery services, hardware infrastructure and maintainance, and payment gateway infrastructure.

1.3 Audience

This document is intended for software developers, system architects, QA engineers, security auditors, and course evaluators responsible for implementing, testing, and assessing the system.

1.4 Definitions and Acronyms

Term / Acronym	Definition
ADR	Architecture Decision Record
API	Application Programming Interface
JWT	JSON Web Token (HTTP-only bearer authorization)
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
SAD	Software Architecture and Design Specification
SRS	Software Requirements Specification
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege

Term / Acronym	Definition
TLS	Transport Layer Security (version 1.3)

2 Document Overview

2.1 How to Use This Document

This SAD document is organized to give stakeholders a top-down view of the system architecture:

- **Section 3 (Architecture):** Focuses on macro-level system decomposition, architectural patterns, component interaction models, technology choices, STRIDE threat models, and requirement traceability.
- **Section 4 (Design):** Focuses on detailed module designs, sequence diagrams for key workflows, REST API endpoint definitions, logging/monitoring standards, and UX guidelines.
- **Section 5 (Appendices):** Provides reference standards, architectural decision records, and tooling references.

2.2 Related Documents

- **Software Requirements Specification (SRS):** Defines the functional (`SYS-F-001..020`), non-functional (`SYS-NF-001..005`), and security (`PRJ-SO-*`, `PRJ-SR-*`) requirements traceable to this architecture.
- **Software Test Plan (STP):** Defines automated test suites (`TS-AUTH`, `TS-CRS`, `TS-ATT`, `TS-GRD`, `TS-PERF`, `TS-SEC`) verifying architectural compliance.
- **Requirements Traceability Matrix (RTM):** Maps SRS requirement IDs to design specifications and test cases.

3 Architecture

3.1 Goals & Constraints

- **Primary Architectural Goals:**
  - High Responsiveness: 95% of API requests processed in < 800 ms under 500 active sessions (`SYS-NF-001`).
  - Concurrency: Support 200 concurrent attendance/grade submission transactions without database deadlock (`SYS-NF-002`).
  - Availability: Maintain 99.0% operational uptime during active terms (`SYS-NF-003`).
  - Security & Confidentiality: Role-based data isolation (`PRJ-SO-001`) and strict workflow state integrity (`PRJ-SO-002`).
- **Architectural Constraints:**
  - Agile Timeline: Deliverable within 2 one-week Agile sprints by a 3-student development team.
  - No SMTP Infrastructure: Initial temporary credentials displayed on administrative UI views (`SYS-F-001`).

– Immutable 48-Hour Freeze: Hard backend time check revoking faculty attendance edits 48h post-session (`SYS-F-010`).

3.2 Stakeholders & Concerns

Stakeholder	Primary Architectural Concerns
Students	Fast response times (< 800 ms), clear UI, grade privacy, transparent regrade dispute tracking.
Faculty	Effortless roll-call (≤ 3 clicks), clear roster visibility, rubric grade submission.
Administrators	System-wide visibility, operational overrides, account management, institution metrics export.
Evaluators / QA	Clean layer separation (3-tier), automated test coverage ($\geq 70\%$), security compliance.

3.3 Component Diagram

The system is structured into cohesive components separated across Presentation, API Gateway, Application Business Logic, and Persistence layers.

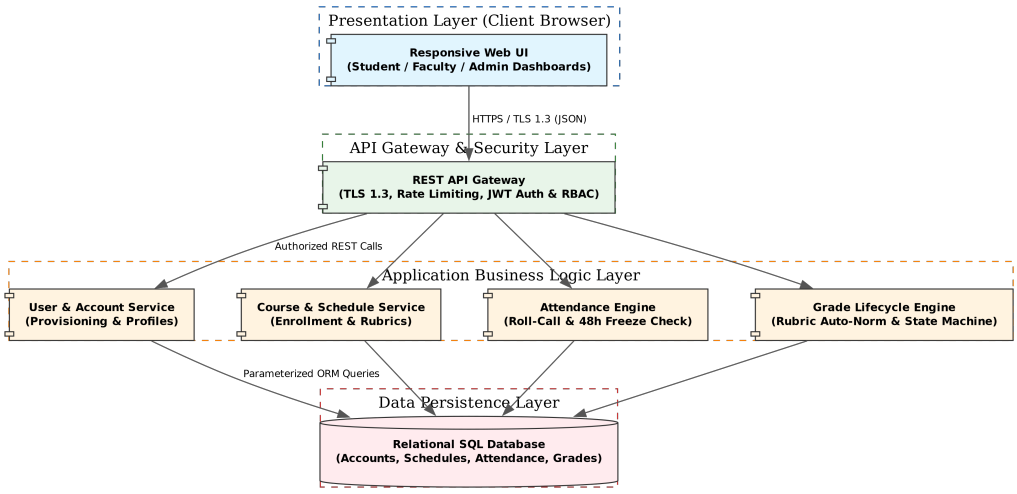


Figure 1: System Component Architecture Diagram

3.4 Component Descriptions

- **Web UI Presentation Component:** Responsive Single Page Application (SPA) providing role-tailored dashboards for Students, Faculty, and Administrators.
- **API Gateway & Auth Middleware:** Handles TLS 1.3 termination, rate limiting, JWT session validation (48h expiry), and RBAC endpoint middleware (`PRJ-SR-001..004`).

- **User & Account Service:** Manages administrative user provisioning, role assignments, temporary credential resets, and contact details updates.
- **Course & Schedule Service:** Handles course offerings, section capacity management, rubric scheme auto-normalization, student self-enrollment (max 6), and class schedule publishing.
- **Attendance Engine:** Manages digitized roll-call entry, enforces the strict 48-hour modification freeze constraint, and processes administrative overrides.
- **Grade Lifecycle Engine:** Manages numeric rubric evaluations, state transitions (`Draft` → `Submitted` → `Under Review` → `Published`), and regrade dispute workflows.
- **Persistence Layer:** Relational database management system with parameterized ORM queries preventing SQL injection (`PRJ-SR-005`).

3.5 Chosen Architecture Pattern and Rationale

The system adopts a **Classic 3-Tier Layered Architecture** (Presentation, Application/Logic, Data Persistence).

- **Rationale:** A monolithic 3-tier layered architecture offers clear separation of concerns, simplifies local deployment, minimizes network hop latency (< 800 ms target), and fits perfectly within the 2-week implementation window of a 3-student team. Microservices were explicitly rejected due to deployment overhead and distributed transaction complexity.

3.6 Technology Stack & Data Stores

- **Frontend:** HTML5, CSS3 (Flexbox/Grid), Modern JavaScript (React).
- **Backend Application Server:** Python (Flask).
- **Database Management System:** MySQL relational database using ORM (Prisma/SQLAlchemy).
- **Security & Auth:** TLS 1.3, bcrypt password hashing (factor 12), HTTP-only JWT bearer tokens.

3.7 Risks & Mitigations

Risk	Impact	Likelihood	Mitigation Strategy
Database connection pool exhaustion during peak roll-calls	High	Medium	Implement connection pooling, index `session_id` and `student_id` fields, load test for 200 concurrent POSTs.
Unauthorized attendance edits after 48 hours	High	Low	Enforce 48-hour timestamp checks strictly in backend API middleware, independent of client UI logic.

Risk	Impact	Likelihood	Mitigation Strategy
Unbalanced rubric component weights	Medium	Low	Implement backend auto-normalization of component weights to sum to 100% on save (`SYS-F-005`).

3.8 Traceability to Requirements

Requirement ID	Component / Module	Design Implementation
`SYS-F-001..004`	User & Account Service	REST endpoints `/api/v1/users`, temporary credential display screen, bcrypt hashing.
`SYS-F-005..008`	Course & Schedule Service	REST endpoints `/api/v1/courses`, self-enrollment cap logic (6 courses), schedule mapping.
`SYS-F-009..012`	Attendance Engine	REST endpoints `/api/v1/attendance`, 48h automated freeze middleware, admin override endpoint.
`SYS-F-013..017`	Grade Lifecycle Engine	REST endpoints `/api/v1/grades`, rubric auto-normalization, state machine handler.
`SYS-F-018..020`	Analytics Service	Consolidated dashboard SQL views, GPA calculation engine, metric CSV exporter.

3.9 Security Architecture (STRIDE Threat Model)

Threat Category	Potential Threat	Architectural Mitigation
Spoofing	User credential theft / Session hijacking	Enforce TLS 1.3 transit encryption (`PRJ-SR-001`), bcrypt hashing (`PRJ-SR-002`), HTTP-only 48h JWTs (`PRJ-SR-003`).

Threat Category	Potential Threat	Architectural Mitigation
Tampering	Parameter tampering of attendance or grade values	Parameterized ORM queries (`PRJ-SR-005`), server-side RBAC validation (`PRJ-SR-004`).
Repudiation	Disputed grade alterations or attendance changes	Rigid state machine transitions (`Draft` → `Published`), administrative override logging.
Information Disclosure	Unauthorized student grade/profile access	RBAC middleware enforcing strict tenant/role data isolation (`PRJ-SO-001`).
Denial of Service	API flooding during attendance roll-call	Rate limiting on API gateway endpoints, connection pooling, performance target < 800 ms.
Elevation of Privilege	Student attempting faculty or admin actions	Role claims embedded in signed JWTs and checked by backend middleware on every request (`PRJ-SR-004`).

4 Design

4.1 Design Overview

The detailed design translates architectural components into specific modules, RESTful API endpoint contracts, sequence interaction models, error handling guidelines, and UI user-experience patterns.

4.2 UML Sequence Diagrams

4.2.1 Sequence Diagram 1: Digitized Attendance Marking & 48-Hour Freeze

The sequence below illustrates a Faculty member executing roll-call and the backend enforcing the 48-hour freeze constraint.

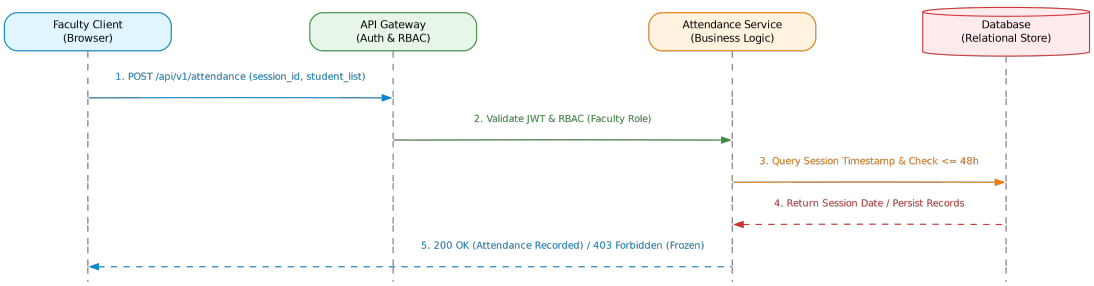


Figure 2: Digitized Attendance Marking & 48-Hour Freeze Sequence Diagram

4.2.2 Sequence Diagram 2: Weighted Rubric Grade Lifecycle

The sequence below illustrates grade progression from `Draft` through `Submitted` and `Published`, including Student regrade disputes (`Under Review`). Each grade can be disputed only once; a re-published grade is final.

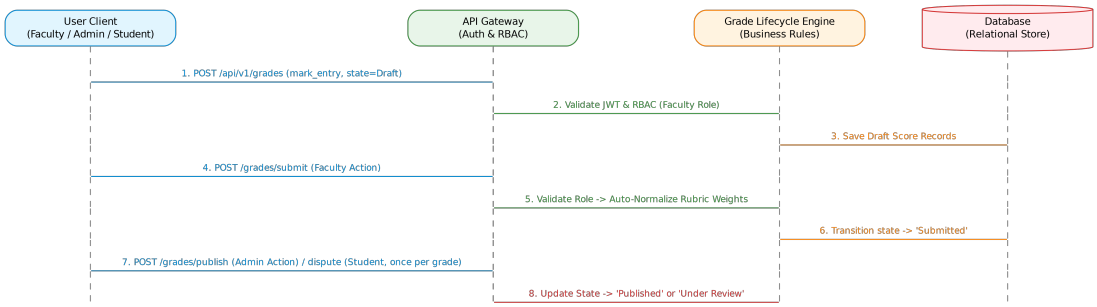


Figure 3: Weighted Rubric Grade Lifecycle Sequence Diagram

4.3 API Design

4.3.1 Component Interface 1: Attendance Marking Endpoint

- **Endpoint:** `/api/v1/attendance/sessions/{sessionId}`
- **Method:** `POST`
- **Headers:** `Authorization: Bearer <JWT>`, `Content-Type: application/json`
- **Request Body:**

```
{
  "session_id": "SESS-2026-101",
  "attendance_list": [
    { "student_id": "PES1UG24CS001", "is_present": true },
    { "student_id": "PES1UG24CS002", "is_present": false }
  ]
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Attendance recorded for 2 students.",
  "frozen_at": "2026-10-14T10:00:00Z"
}
```

- **Error Responses:**
  - `401 Unauthorized`: Missing or expired JWT token.
  - `403 Forbidden`: Attendance session is frozen (> 48 hours post-session start).
  - `404 Not Found`: Invalid session ID or student not enrolled in section.

4.3.2 Component Interface 2: Grade Lifecycle & Dispute Endpoint

- **Endpoint:** `/api/v1/grades/courses/{courseId}/students/{studentId}`
- **Method:** `PUT`
- **Headers:** `Authorization: Bearer <JWT>`, `Content-Type: application/json`
- **Request Body:**

```
{
  "rubric_scores": [
    { "rubric_id": "RUB-01", "score": 85.0 },
    { "rubric_id": "RUB-02", "score": 90.0 }
  ],
  "target_state": "Submitted"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
```

```
  "weighted_total": 87.5,
  "grade_state": "Submitted"
}
```

- **Error Responses:**
  - `400 Bad Request`: Rubric scores out of valid range (0-100).
  - `403 Forbidden`: User role unauthorized for target state transition.

4.4 Error Handling, Logging & Monitoring

- **Error Handling:** Standardized REST error envelopes containing HTTP status code, machine-readable error codes, and human-readable messages.
- **Logging Standards:** Structured JSON logging (Python `logging`). Sensitive data (passwords, JWT tokens, personal contact details) are strictly sanitized and excluded from log streams.
- **Monitoring Metrics:** API latency metrics (tracking < 800 ms threshold), error rates (5xx responses), and concurrent transaction counts.

4.5 UX Design

- **Design System:** Clean, accessible web interface compliant with WCAG 2.1 AA accessibility guidelines.
- **Navigation Efficiency:** Key user workflows (taking roll-call, viewing student GPA) are reachable within ≤ 3 clicks from login.
- **Responsive Layout:** Optimized for devices ranging from mobile screens (375px) to desktop monitors (1920px).

4.6 Open Issues & Next Steps

- **Future Work:** Evaluate automated batch notification options for dispute status updates.
- **Scalability Enhancements:** Implement Redis cache layer for course catalog queries if institution size exceeds 10,000 active students.

5 Appendices

5.1 Glossary

Term	Definition
ADR	Architecture Decision Record
API	Application Programming Interface
bcrypt	Adaptive cryptographic hash function for passwords
JWT	JSON Web Token used for session authorization
ORM	Object-Relational Mapping library
RBAC	Role-Based Access Control

Term	Definition
SAD	Software Architecture and Design Specification
STRIDE	Threat modeling framework (Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege)
TLS	Transport Layer Security encryption protocol

5.2 References

- **IEEE Std 42010-2011:** Systems and software engineering — Architecture description.
- **OWASP Top 10 Web Application Security Risks:** Security guidelines for SQLi, XSS, and Broken Access Control.
- **NIST SP 800-63B:** Digital Identity Guidelines & Password Management Recommendations.

5.3 Tools

- **Pandoc & XeLaTeX:** Markdown-to-PDF document build pipeline.
- **Graphviz:** Structural diagram rendering engine (`.dot` to 300 DPI `.png`).
- **OpenAPI / Swagger:** RESTful API interface specification.